

CardsFlow — Frontend Architecture & Backend Handoff Guide

Target Audience: Lead Backend / Full-Stack Developer **Project:** CardsFlow Platform
([zoromba86/cardsflow](#)) **Date:** April 2026

1. Executive Summary

This document serves as the technical bridge between the newly completed frontend repository and the upcoming backend integration. The frontend has been entirely rebuilt and optimized for a premium, institutional fintech aesthetic. It achieves a Lighthouse performance score >90 while employing complex 3D glassmorphism and scroll-triggered animations.

Your primary objective is to wire up the existing UI components to your backend APIs (Authentication, Payment Processing, Card Issuance, and ZK Verification) and deploy the unified application to production.

2. Technology Stack & Library Differences

If you compare the codebase to previous iterations, note the following strict library choices made to enforce performance and security:

- **Framework: Next.js 16 (App Router)** with **React 19**. We are utilizing React Server Components (RSC) where possible, reserving `"use client"` only for animated or interactive components.
- **Styling: Tailwind CSS v4** (using the new `@theme` API in `globals.css`). We strictly avoided heavy UI frameworks like Material-UI or Chakra to keep bundle sizes minimal.
- **Accessible Components: Radix UI** primitives (e.g., `@radix-ui/react-accordion`) are used for complex interactive components (like the FAQ) to ensure WAI-ARIA compliance without shipping default styles.

- **Animations: Framer Motion** (`framer-motion@12`).
 - *Important Technical Note:* We previously encountered severe main-thread scroll lag due to SVG filters (`feTurbulence` , `feDisplacementMap`). We completely refactored these to use GPU-accelerated CSS conic-gradients and Framer Motion `transform` / `opacity` transitions. **Do not reintroduce SVG filter animations.**
-

3. Security & Compliance Architecture

The frontend is pre-configured with strict security measures required for a financial product. You must maintain these during integration:

1. Middleware & CSP (`src/middleware.ts`)

- Every request generates a unique cryptographic nonce.
- A strict `Content-Security-Policy` (CSP) is injected. Inline scripts are blocked; only scripts matching the nonce can execute.
- *Action Required:* If you integrate third-party SDKs (e.g., Stripe.js, Plaid, or support widgets), you must add their domains to the `script-src` and `connect-src` directives in `middleware.ts` .

2. HTTP Security Headers

- Next.js configuration applies `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , and `Strict-Transport-Security` .

3. Data Minimization (Zero-Knowledge)

- The UI and legal copy reflect a ZK-proof model. The frontend expects the verification callback to return a cryptographic eligibility token, *not* raw PII or document scans.
-

4. State Management & API Integration Pattern

The frontend currently uses static mock data arrays (e.g., `ARTICLES` , `USE_CASES`).

Recommended Integration Approach:

1. **Data Fetching:** For public pages (Blog, Use Cases), replace static arrays with Server Components fetching directly from your database or CMS (`fetch('api...', { next: { revalidate: 3600 } })`).

2. **Client State (Dashboard):** For authenticated user states (Wallet Balance, Card Limits, Transaction History), use React Server Components to pass initial state, and a lightweight library like `SWR` or `React Query` for polling/mutations. We did not include Redux; context or SWR is sufficient.
-

5. Integration Roadmap (The "Rest Steps")

Here is the exact sequence of tasks required to make the platform fully operational:

Step 1: Authentication Engine

- **Target:** `/register` and `/login` pages.
- **Action:** Connect UI forms to your Auth provider (custom JWT, NextAuth, or Supabase).
- **Requirement:** Enforce 2FA enrollment upon first login. Store the JWT securely (HTTP-only cookies preferred over `localStorage` due to XSS risks).

Step 2: ZK Verification Callback

- **Target:** Post-registration dashboard gate.
- **Action:** Implement the callback endpoint that receives the Zero-Knowledge proof from the verification partner and updates the user's `eligibility_status` in the DB.

Step 3: Wallet Funding & Ledger

- **Target:** Dashboard "Top Up" flow.
- **Action:** Integrate the crypto/fiat payment gateway (e.g., NowPayments/Stripe).
- **Requirement:** The UI enforces a \$60 minimum first-deposit. You must validate this threshold on the backend before enabling card issuance.

Step 4: Card Issuance API (Visa BIN Sponsor)

- **Target:** Dashboard "Issue Card" flow.
- **Action:** Map the frontend form (Card Name, Spend Limit, Allowed Categories) to your card issuer's API.

- **Requirement:** Deduct the \$5 (virtual) or \$50 (physical) issuance fee from the user's ledger atomically during the API call.

Step 5: Webhook & Transaction Sync

- **Target:** Dashboard Transaction History.
 - **Action:** Listen for authorization webhooks from the card network. Calculate and deduct the tier-based top-up fees (5%-7%) and foreign exchange fees (1.2%) programmatically.
-

6. Deployment Instructions (Ubuntu / Node.js VPS)

Since the target environment is an Ubuntu VPS, follow these steps to deploy the Next.js application behind Nginx.

1. Clone & Build

```
git clone https://github.com/zoromba86/cardsflow.git /var/www/cardsflow
cd /var/www/cardsflow
npm ci
npm run build
```

2. Process Management (PM2) Do not run `npm start` directly in tmux/screen. Use PM2 to manage the process and enable auto-restart on crashes/reboots.

```
sudo npm install -g pm2
pm2 start npm --name "cardsflow-frontend" -- start
pm2 save
pm2 startup
```

3. Nginx Reverse Proxy & HSTS Create an Nginx server block to terminate SSL and proxy traffic to port 3000. *CRITICAL:* Ensure you manually add the HSTS header in Nginx, as hosting environments often strip it from the Next.js response.

```
server {
    listen 443 ssl http2;
    server_name cardsflow.net;
```

```
# Let's Encrypt SSL config here

# Enforce HSTS (Required for Security Rating)
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload"
always;

location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
}
}
```

4. Environment Variables

Ensure `.env.production` is populated with absolute URLs:

```
NEXT_PUBLIC_BASE_URL=https://cardsflow.net
NEXT_PUBLIC_API_URL=https://api.cardsflow.net/v1
```

End of Document. For any architectural clarifications, refer to the component structure in `src/components/blocks` and `src/components/ui`.